

**A Report
on
Movie Ticket Booking System**

Submitted by

T.Naga varshitha lakshmi- AP23110011136

SK.Farzana - AP23110011149

Bhargavi Bharothu - AP23110011484

Aparna Medida - AP23110011528

Under the Guidance of

Syrd Arshad

**BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING**



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SRM UNIVERSITY-AP NEERUKONDA

NEERUKONDA MANAGALAGIRI - 522503

ANDHRA PRADESH, INDIA

May -2026

INTRODUCTION:

Movie Ticket Booking System is a versatile and user-friendly software solution designed to simplify the process of booking cinema tickets online. Suitable for individuals, families, and group bookings, it provides an easy-to-use interface where users can browse movies, check show timings, and select preferred seats in real time. The system offers a variety of features, including seat selection, multiple payment options, booking history, and instant ticket confirmation via email or SMS. Users can also filter movies based on language, genre, or location, and view trailers and ratings before booking.

The platform ensures secure transactions through advanced security measures such as encrypted payment gateways and user authentication. It can seamlessly integrate with third-party services like payment providers and notification systems to automate confirmations and reminders. Additionally, built-in analytics tools help theater owners track bookings, manage occupancy, and generate reports for better decision-making. With its flexibility, convenience, and efficient performance, the Movie Ticket Booking System is an ideal solution for cinemas and customers looking for a smooth and hassle-free ticket reservation experience.

SCENARIO Based Case Study:

Meet Alex, a busy professional who enjoys watching movies but struggles to find time to stand in long queues at theaters. With his tight schedule, Alex prefers a quick and convenient way to book movie tickets, choose his favorite seats, and stay updated on the latest releases.

Alex discovers an Online Movie Ticket Booking System, a comprehensive web application designed to simplify and enhance the movie ticket booking experience. Curious about its features, Alex decides to explore how it can make his movie outings more convenient and enjoyable.

User Registration and Authentication:

Alex registers on the platform by providing his basic details and preferences. He logs in securely using his credentials, ensuring that his personal and payment information remains safe and protected.

Movie Catalog and Show Details:

Alex browses through a wide range of movies available on the platform. He can view details such as movie genres, ratings, trailers, and show timings across different theaters. This helps him make informed decisions about which movie to watch.

Seat Selection and Booking:

The system provides an interactive seating layout, allowing Alex to choose his preferred seats

easily. He can view seat availability in real time and select seats that suit his comfort and preference.

Payment and Confirmation:

Alex proceeds to make the payment using various options like credit/debit cards, UPI, or digital wallets. Once the payment is successful, he receives instant confirmation of his booking via email or SMS, along with a digital ticket.

Booking Management:

The platform allows Alex to view his booking history, cancel tickets if necessary, or modify his plans based on availability. This flexibility helps him manage his movie plans efficiently.

Notifications and Reminders:

The system sends timely notifications about upcoming shows, booking confirmations, and reminders before the movie starts, ensuring Alex never misses his show.

Offers and Loyalty Programs:

The platform provides Alex with access to special discounts, cashback offers, and loyalty rewards, enhancing his overall experience and encouraging frequent usage.

Customer Support:

If Alex faces any issues, he can access customer support, FAQs, or help resources available on the platform to resolve his queries quickly.

Alex's Experience:

With the Online Movie Ticket Booking System, Alex can now plan his movie outings effortlessly. He saves time, avoids long queues, and enjoys a seamless booking experience. The platform becomes an essential tool for his entertainment needs, helping him relax and enjoy movies without any hassle.

SYSTEM REQUIREMENTS:

To successfully design, develop, and operate the Movie Ticket Booking Web Application, a set of essential software tools and technologies is required. These components ensure the system runs efficiently, supports scalability, and delivers a smooth user experience.

1. Software Requirements

The following tools and platforms are needed for building and maintaining the application:

- **Operating System:** Windows 11, Ubuntu Linux, or macOS — Enables developers to work in a flexible and compatible environment across platforms.
- **Python (v3.10 or above):** Used as the backend programming language to manage server-side logic, including user authentication, booking processing, and data handling.
- **Django / Flask:** Web frameworks that help in building secure and scalable backend services, including APIs for movie listings, ticket booking, and payment integration.
- **HTML5, CSS3, JavaScript:** Core web technologies used to design and develop the structure, styling, and interactivity of the user interface.
- **Bootstrap / Tailwind CSS:** Frontend frameworks that help create responsive and visually appealing layouts for movie browsing and seat selection pages.
- **MySQL / PostgreSQL:** Relational database systems used to store structured data such as user accounts, movie schedules, booking details, and transaction records.
- **Web Browser:** Latest versions of Microsoft Edge, Google Chrome, or Firefox — Used for accessing, testing, and debugging the application.
- **REST API Tools (Insomnia / Thunder Client):** Used to test and validate API endpoints during development.
- **Code Editor (Sublime Text / IntelliJ IDEA):** Development tools that provide features like code completion, debugging, and project management.
- **Version Control (Git & GitHub):** Helps manage code changes, collaboration, and version tracking throughout the development lifecycle.

2. Hardware Requirements

To ensure the efficient development, deployment, and usage of the Movie Ticket Booking Web Application, the following hardware components are required. These specifications help maintain smooth performance for both developers and end users.

- **Processor (CPU):** Minimum Intel Core i3 / AMD Ryzen 3 or higher — Ensures smooth execution of development tools, server processes, and user operations like booking and payments.
- **RAM:** At least 8 GB (16 GB recommended) — Supports multitasking such as running the database, backend server, and frontend simultaneously without performance lag.
- **Storage:** Minimum 256 GB SSD — Provides faster data access, quick application loading, and efficient storage of movie data, user records, and booking history.
- **Display:** Standard HD screen (1366×768 or higher) — Ensures clear visualization of the user interface, including seat layouts and booking dashboards.
- **Network Connectivity:** Stable internet connection (minimum 10 Mbps) — Required for real-time ticket booking, payment processing, and syncing data with servers.
- **Input Devices:** Keyboard and mouse (or touchpad) — Necessary for navigation, development tasks, and user interactions within the application.
- **Server Hardware (for deployment):** Dedicated or cloud-based server with scalable resources — Handles multiple user requests, manages concurrent bookings, and ensures high availability of the system.

These hardware requirements ensure that the Movie Ticket Booking System operates efficiently, providing a fast, reliable, and seamless experience for both developers and users.

PROJECT ARCHITECTURE:

The Movie Ticket Booking Web Application follows a **multi-tier architecture** that separates concerns into distinct layers, ensuring scalability, maintainability, and efficient performance. This structured approach allows smooth interaction between users, application logic, and data storage.

1. Presentation Layer (Frontend)

This layer is responsible for user interaction and visual representation of the system.

It provides an intuitive interface where users can browse movies, select show timings, choose seats, and make payments.

Technologies such as HTML, CSS, JavaScript, and modern frameworks are used to create a responsive and user-friendly design.

2. Application Layer (Backend)

The backend acts as the core processing unit of the system.

It handles business logic such as user authentication, movie management, seat availability checks, booking processing, and payment integration.

This layer ensures that all user requests are processed correctly and securely through APIs and server-side logic.

3. Database Layer

This layer is responsible for storing and managing all application data.

It includes information such as user profiles, movie details, theater schedules, seat layouts, and booking records.

The database ensures data consistency, quick retrieval, and secure storage of sensitive information.

4. API Layer

The API layer acts as a communication bridge between the frontend and backend.

It allows seamless data exchange using RESTful services, enabling operations like fetching movie listings, submitting bookings, and updating seat availability in real time.

5. Payment Gateway Integration

This component handles secure online transactions.

It connects the application with external payment services to process payments through cards, UPI, or digital wallets while ensuring encryption and transaction safety.

6. Notification Service

This module is responsible for sending booking confirmations, alerts, and reminders to users via email or SMS.

It ensures users stay informed about their bookings and upcoming shows.

7. Admin Panel

The admin interface allows theater owners or administrators to manage movies, schedules, pricing, and bookings.

It also provides access to analytics and reports for better decision-making.

TECHNICAL ARCHITECTURE:

The Movie Ticket Booking Web Application is designed using a **modular and service-oriented technical architecture** that ensures high performance, reliability, and scalability. It focuses on how different technologies and components interact to deliver a seamless booking experience.

1. Client-Side (User Interface Layer)

The client-side is responsible for rendering the application in the user's browser or mobile device.

It handles user interactions such as browsing movies, selecting seats, and initiating bookings.

Dynamic content updates, form validations, and UI responsiveness are managed at this level to provide a smooth experience.

2. Web Server Layer

The web server receives requests from users and forwards them to the appropriate backend services.

It manages routing, handles static content delivery, and ensures secure communication using HTTPS protocols.

3. Application Services Layer

This layer contains the core business logic of the system.

It processes operations such as user authentication, movie scheduling, seat allocation, booking confirmation, and cancellation.

It also ensures that no double booking occurs by managing seat-locking mechanisms in real time.

4. Database Management Layer

This layer manages persistent data storage.

It stores user information, movie listings, theater details, seat availability, booking records, and transaction history.

Efficient indexing and query handling ensure quick data retrieval and updates.

5. Payment Processing Layer

This component integrates with external payment services to handle transactions securely.

It supports multiple payment methods and ensures encrypted communication, transaction validation, and status tracking.

6. Caching Layer

A caching mechanism is used to store frequently accessed data such as movie listings and show timings.

This reduces database load and improves response time for users.

7. Notification & Messaging Layer

This layer manages communication with users by sending booking confirmations, reminders, and alerts via email or SMS.

It ensures reliable message delivery through integrated messaging services.

8. Security Layer

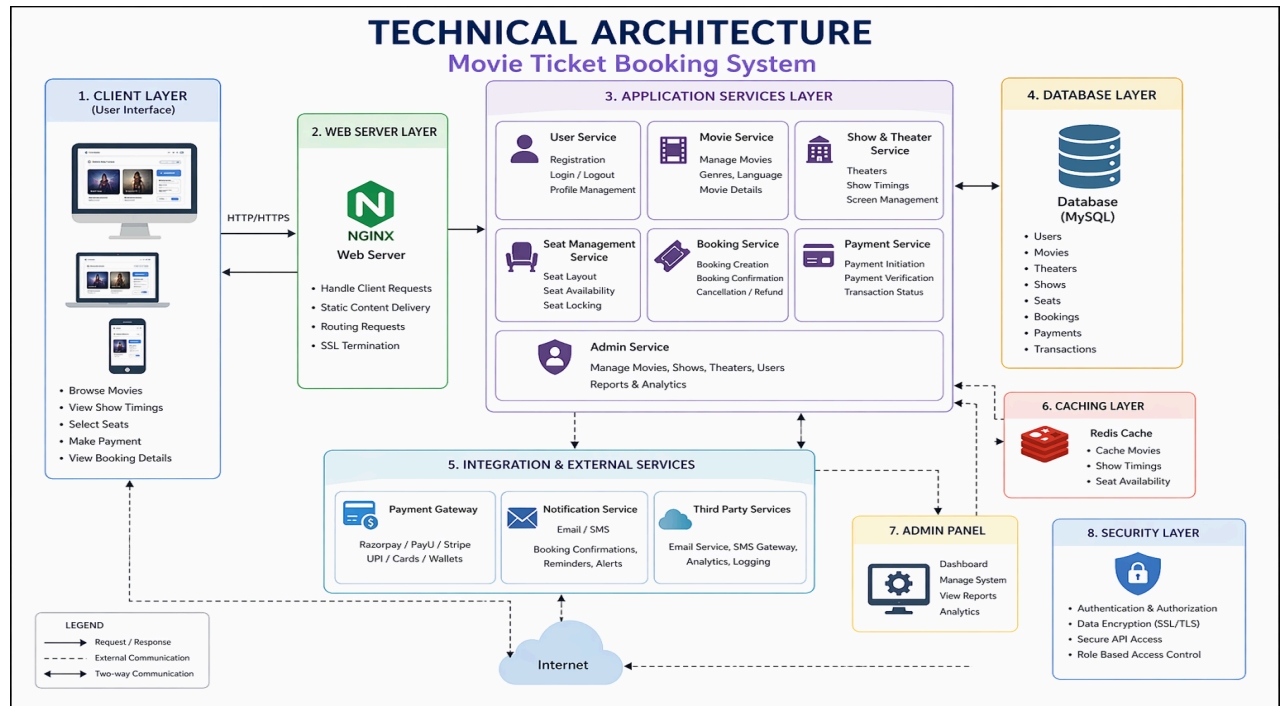
Security is implemented across all layers using authentication, authorization, data encryption, and secure API access.

It protects user data, prevents unauthorized access, and ensures safe transactions.

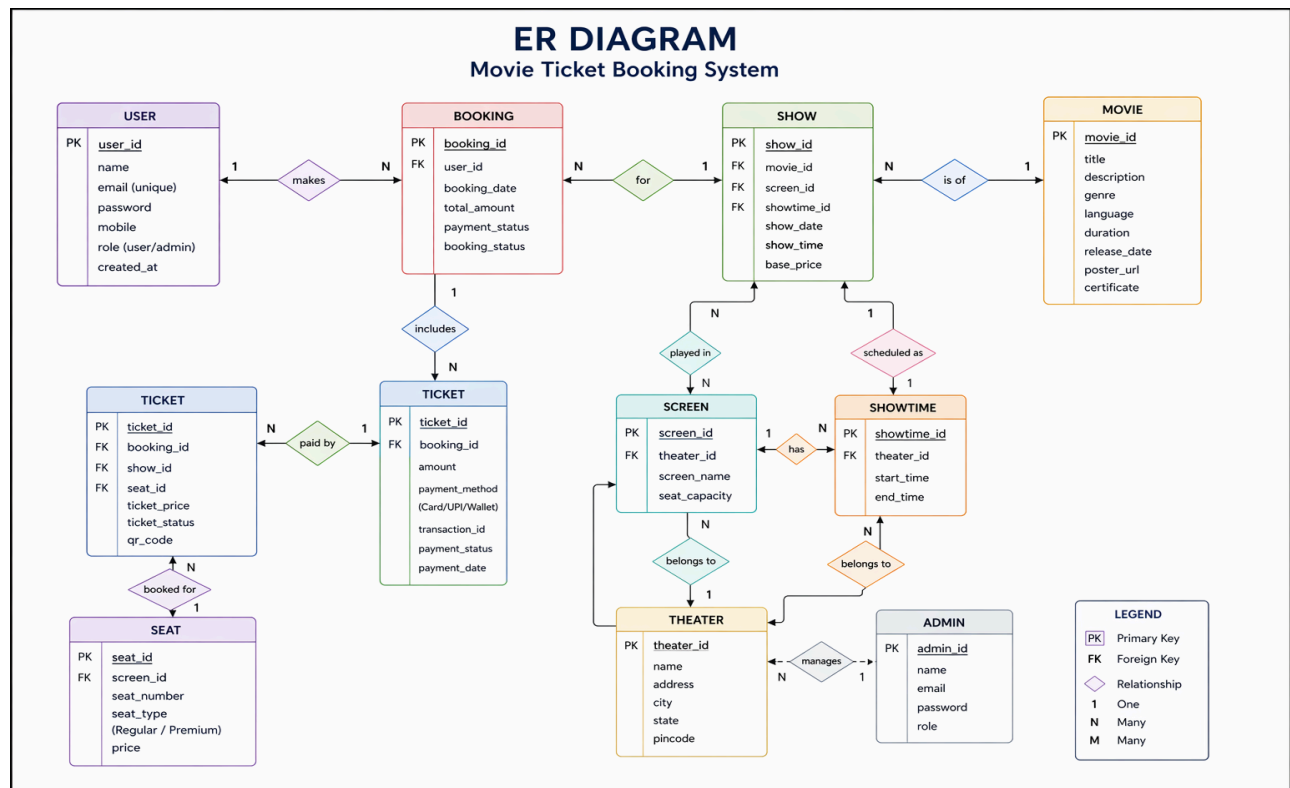
9. Deployment Environment

The application can be deployed on cloud platforms or dedicated servers.

It supports scalability through load balancing and distributed services to handle high user traffic during peak booking times.



ER DIAGRAM:



ER DIAGRAM DESCRIPTION: Movie Ticket Booking System

The Entity-Relationship (ER) diagram represents the structure of the Movie Ticket Booking System database. It illustrates the entities, their attributes, and the relationships between them.

1. Entities and Attributes

USER

Represents customers and administrators using the system.

Attributes:

- user_id (Primary Key)
- name
- email (unique)
- password
- mobile
- role (customer/admin)
- created_at

BOOKING

Stores details of ticket bookings made by users.

Attributes:

- booking_id (Primary Key)
- user_id (Foreign Key)
- booking_date
- total_amount
- payment_status
- booking_status

TICKET

Represents individual tickets generated for a booking.

Attributes:

- ticket_id (Primary Key)
- booking_id (Foreign Key)
- seat_id
- ticket_price
- ticket_status
- qr_code

SEAT

Contains details about seats on a screen.

Attributes:

- seat_id (Primary Key)

- screen_id (Foreign Key)
- seat_number
- seat_type (Regular/Premium)
- price

MOVIE

Stores information about movies available for booking.

Attributes:

- movie_id (Primary Key)
- title
- description
- genre
- language
- duration
- release_date
- poster_url
- certificate

SHOW

Represents a specific movie show at a particular time.

Attributes:

- show_id (Primary Key)
- movie_id (Foreign Key)
- screen_id (Foreign Key)
- showtime_id (Foreign Key)
- show_date
- show_time
- base_price
- available_seats

SCREEN

Represents screens within a theater.

Attributes:

- screen_id (Primary Key)
- theater_id (Foreign Key)
- screen_name
- seat_capacity

SHOWTIME

Stores timing details for shows.

Attributes:

- showtime_id (Primary Key)

- theater_id (Foreign Key)
- start_time
- end_time

THEATER

Represents cinema theaters.

Attributes:

- theater_id (Primary Key)
- name
- address
- city
- state
- pincode

ADMIN

Represents system administrators.

Attributes:

- admin_id (Primary Key)
- name
- email
- password
- role

2. Relationships

- **USER → BOOKING (1:N)**
A user can make multiple bookings.
- **BOOKING → TICKET (1:N)**
One booking can include multiple tickets.
- **TICKET → SEAT (N:1)**
Each ticket is assigned to one seat.
- **BOOKING → SHOW (N:1)**
Each booking is associated with one show.
- **MOVIE → SHOW (1:N)**
A movie can have multiple shows.
- **SHOW → SCREEN (N:1)**
Each show is conducted on one screen.
- **SCREEN → THEATER (N:1)**
Multiple screens belong to one theater.
- **SHOW → SHOWTIME (N:1)**
Each show is scheduled at a specific time.
- **ADMIN → THEATER (1:N)**
Admin manages one or more theaters.

Key Features:

The Movie Ticket Booking System offers a range of features designed to provide a smooth, efficient, and user-friendly experience for both customers and administrators.

1. User Registration & Login

Users can easily create an account and log in securely to access booking services, view history, and manage their profiles.

2. Movie Browsing & Search

Users can explore available movies, view details such as genre, language, ratings, and trailers, and search based on preferences.

3. Real-Time Show Availability

Displays updated show timings, theater locations, and seat availability, ensuring users get accurate information before booking.

4. Seat Selection System

Provides an interactive seating layout where users can choose their preferred seats (Regular/Premium) in real time.

5. Online Ticket Booking

Allows users to book tickets instantly without standing in queues, making the process fast and convenient.

6. Multiple Payment Options

Supports secure payment methods such as debit/credit cards, UPI, and digital wallets for easy transactions.

7. E-Ticket Generation

Generates digital tickets with QR codes that can be used for entry at the theater.

8. Booking History & Management

Users can view past bookings, track current reservations, and cancel tickets if required.

9. Notifications & Alerts

Sends booking confirmations, reminders, and updates via email or SMS to keep users informed.

10. Admin Dashboard

Admins can manage movies, shows, theaters, and bookings through a centralized control panel.

11. Reports & Analytics

Provides insights into ticket sales, user activity, and occupancy rates to support better decision-making.

12. Secure System

Implements authentication, data encryption, and secure payment gateways to protect user data and transactions.

ROLES AND RESPONSIBILITIES:

The system involves different types of users, each with specific roles and responsibilities to ensure smooth operation and management.

1. Customer (User)

Customers are the primary users of the system who book movie tickets.

Responsibilities:

- Register and log in to the system
- Browse movies, show timings, and theaters
- Select seats and book tickets
- Make online payments securely
- View and manage booking history
- Cancel or modify bookings (if allowed)
- Receive notifications and confirmations

2. Admin (Administrator)

Admins manage and control the overall system operations.

Responsibilities:

- Add, update, or remove movies from the system
- Manage theater details, screens, and seat layouts
- Schedule show timings for movies
- Monitor bookings and transactions
- Generate reports and analyze system performance
- Handle user issues and provide support
- Ensure system security and data integrity

User Flow:

Customer (User)

Registration and Login



Browse Movies & Show Timings



Select Theater & Seats



Ticket Booking



Payment



E-Ticket Confirmation (SMS/Email)



Notifications & Reminders

Admin

Movie & Show Management



Theater & Screen Management



User Management



Booking & Payment Monitoring



Reports & Analytics

3. Theater Manager

Responsible for managing specific theater operations.

Responsibilities:

- Manage screens and seating arrangements
- Update show schedules and pricing
- Monitor seat availability and occupancy
- Coordinate with admin for movie listings
- Handle on-site operations and customer queries

4. Payment Gateway (External System)

Handles secure transaction processing.

Responsibilities:

- Process online payments (cards, UPI, wallets)
- Verify transactions and update payment status
- Ensure secure and encrypted payment handling
- Notify system about payment success or failure

5. System (Application)

The software system itself plays a crucial role in automation.

Responsibilities:

- Authenticate users and manage sessions
- Process booking requests and prevent double booking
- Maintain database records (users, movies, bookings)
- Generate tickets and QR codes
- Send notifications and alerts
- Ensure system performance and reliability

MVC PATTERN:

The Movie Ticket Booking System follows the Model–View–Controller (MVC) architectural pattern, which separates the application into three interconnected components. This separation improves code organization, scalability, and maintainability.

1. Model (Data Layer)

The Model represents the data and business logic of the application. It manages how data is stored, retrieved, and updated.

Responsibilities:

- Handles database operations (CRUD: Create, Read, Update, Delete)
- Maintains entities such as Users, Movies, Bookings, Seats, and Payments
- Applies business rules like seat availability and booking validation
- Ensures data consistency and integrity

Example:

When a user books a ticket, the model updates seat availability and stores booking details in the database.

2. View (Presentation Layer)

The View is responsible for the user interface and how data is displayed to users.

Responsibilities:

- Displays movie listings, show timings, and seat layouts
- Provides forms for login, registration, and booking
- Shows booking confirmations and error messages
- Ensures a responsive and user-friendly design

Example:

The seat selection page where users visually choose available seats is part of the View.

3. Controller (Logic Layer)

The Controller acts as an intermediary between the Model and the View. It processes user input and updates the Model and View accordingly.

Responsibilities:

- Handles user requests (login, booking, payment)
- Validates user input
- Calls Model functions to fetch or update data
- Sends processed data to the View for display

Example:

When a user selects seats and clicks “Book Now,” the controller processes the request, checks seat availability via the Model, and updates the View with booking confirmation.

Working Flow of MVC:

1. The user interacts with the View (e.g., selects a movie).
2. The Controller receives the request and processes it.
3. The Model handles data operations and business logic.
4. The result is sent back to the View for display to the user.

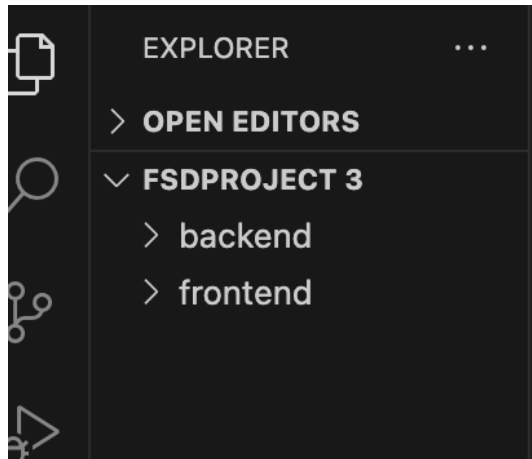
Advantages of MVC in This System:

- Separation of concerns for better code management
- Easy to update UI without affecting backend logic
- Improved scalability and maintainability
- Supports team development (frontend & backend work independently)

Project Setup And Configuration

Creating project folder

1. Create a new folder with your <project name>.
2. Inside that folder create two new folders.
3. Name one as Client
4. Name another one as Server
5. Now open that folder in VS Code.



Client setup (installing react app)

Open the Client folder in the terminal of VScode.

- `npm create vite@latest .`
- `-- --template react`

Select React framework from the given options.

- Select a framework:
- | React

Select JavaScript variant from the given options.

- Select a variant:
- | JavaScript

Now lets navigate to the client folder by giving the following command.

- `cd client`

To install all the packages run the following command.

- `npm install`

To start the React server type the following command.

- npm run dev

Server setup (npm init)

Open Server folder in terminal of VScode.

- npm init -y

Create files:

- server.js

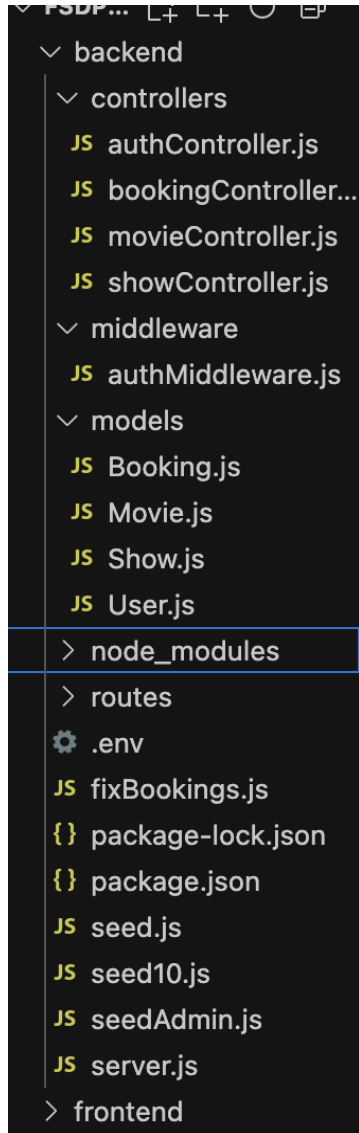
Create folders:

- models
- controllers
- Routes

BACKEND DEVELOPMENT:

Backend Reference Video:

https://drive.google.com/file/d/1_X0wl9vayaRFkc0JWo600xeev3LYcrlf/view?usp=drivesdk



Controllers

The **controllers** folder contains the core business logic of the application. Each controller handles incoming requests, processes data, interacts with models, and sends appropriate responses to the client.

1. authController.js

This controller manages user authentication and authorization functionalities.

- Handles **user registration and login**
- Generates and verifies **JWT tokens**
- Ensures secure access to protected routes
- Manages password validation and encryption

2. bookingController.js

This controller is responsible for handling booking-related operations.

- Creates new movie ticket bookings
- Fetches user booking details
- Updates or cancels bookings
- Ensures seat availability before confirming booking

3. movieController.js

This controller manages movie-related data.

- Adds new movies to the system
- Retrieves movie details
- Updates and deletes movie records
- Displays available movies for users

4. showController.js

This controller handles show scheduling and management.

- Creates and manages movie shows
- Handles show timings and availability
- Retrieves show details based on movies
- Updates or deletes show information

Models

The **models** folder defines the structure of the data stored in the database. Each model represents a collection and is responsible for interacting with the database (usually using MongoDB with Mongoose).

1. Booking.js

This model handles booking-related data.

- Stores details of movie ticket bookings
- Includes user information, movie details, show timing, and seat numbers
- Tracks booking status (confirmed/cancelled)
- Maintains booking date and transaction details

2. Movie.js

This model represents movie data in the system.

- Stores movie name, genre, duration, and language
- Includes movie description and release date
- Maintains poster or image URL
- Helps in displaying available movies to users

3. Show.js

This model manages show-related information.

- Stores show timings for movies
- Links movies with specific screens/theaters
- Maintains seat availability
- Helps in scheduling and managing different shows

4. User.js

This model handles user information.

- Stores user details like name, email, and password
- Manages user roles (admin/user)
- Keeps track of user bookings
- Ensures password security (hashed passwords)

Middleware

The system uses authentication and authorization middleware to protect routes and ensure that only valid users can access specific features. This is implemented in the file `authMiddleware.js`.

`authMiddleware.js` Used to verify whether a user is logged in or not

- Implemented in the file: `authMiddleware.js`

Main Function: `protect`

- Checks for token in request header (Authorization)
- Expects format: Bearer Token
- Verifies token using JWT
- Extracts user information from token
- Stores user data in `req.user`
- Allows request if token is valid
- Returns 401 Unauthorized if token is missing or invalid

Purpose

- Secures the application
- Controls access to protected routes

1. Protect Middleware (`protect`)

This middleware is used to verify whether a user is logged in.

How it works:

- It checks the Authorization header from the request
- Looks for a token starting with “**Bearer**”
- Extracts the token and verifies it using JWT
- If valid → user details are attached to `req.user`
- If invalid → returns error (401 Unauthorized)

Purpose:

- Secures routes like booking tickets, viewing bookings, and payments

- Prevents unauthorized access

2. Admin Middleware (admin)

This middleware checks whether the logged-in user is an admin.

How it works:

- Checks if req.user exists (from protect middleware)
- Verifies req.user.isAdmin
- If true → access granted
- If false → access denied (401 Unauthorized)

Purpose:

- Restricts admin-only features like:
 - Adding movies
 - Managing shows
 - Viewing reports

ROUTES

1. authRoutes.js

- Handles user authentication
- Used for login and registration

Functions:

- User registration (create account)
- User login (generate token)
- Logout (optional)

Purpose:

- Allows users to access the system securely
- Generates JWT token for protected routes

2. bookingRoutes.js

- Manages ticket booking operations

Functions:

- Create new booking
- View user bookings
- Cancel booking
- Get booking details

Purpose:

- Handles complete ticket reservation process
- Connects user, show, and seats

3. movieRoutes.js

- Manages movie-related data

Functions:

- Add movie (admin)
- Update movie details
- Delete movie
- View all movies

Purpose:

- Controls movie listings in the system
- Helps users browse available movies

4. showRoutes.js

- Handles show scheduling

Functions:

- Create show (admin)
- Update show timings
- Delete show
- View available shows

Purpose:

- Manages movie show timings and availability
- Connects movies with screens and time slots

DATABASE DEVELOPMENT:

Create Schemas:

Firstly, configure the Schemas for MongoDB database, to store the data in such a pattern. Use the data from the ER diagrams to create the schemas. The Schemas for This application looks like the one provided below.

1) Users.js → Schema/model for user accounts.

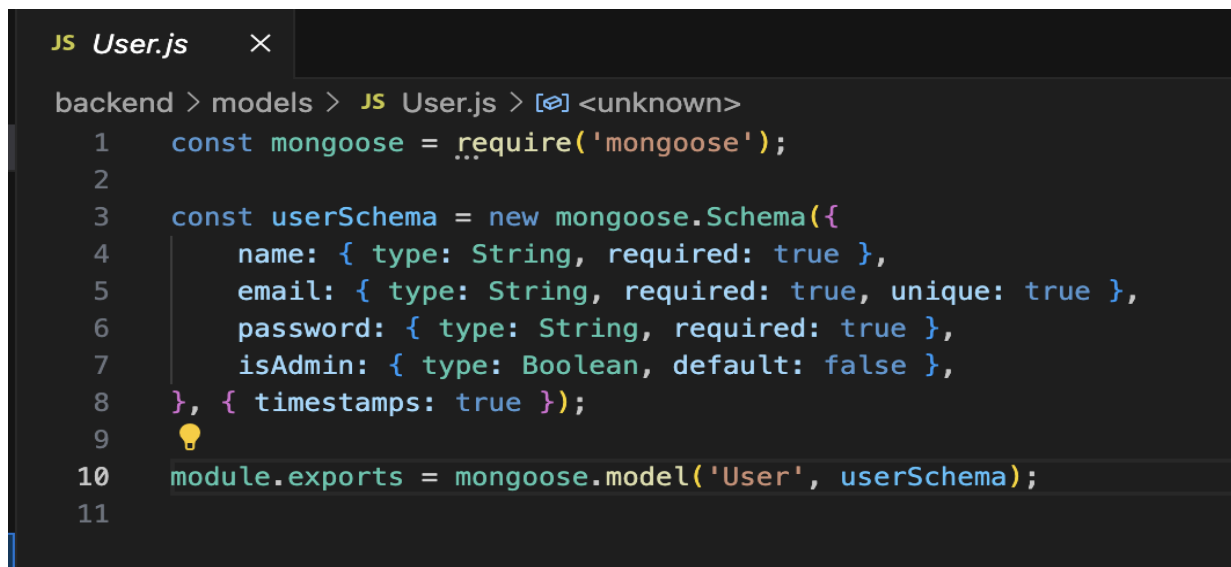
Purpose

The User model is used to:

- Store user information securely
- Handle authentication and authorization
- Manage user-related activities in the system

Common Fields

- **name** – Stores the user's name
- **email** – Stores unique user email
- **password** – Stores encrypted password
- **role** – Defines user type (user/admin)
- **createdAt** – Stores account creation date



```
JS User.js ×
backend > models > JS User.js > [?] <unknown>
1  const mongoose = require('mongoose');
2
3  const userSchema = new mongoose.Schema({
4    name: { type: String, required: true },
5    email: { type: String, required: true, unique: true },
6    password: { type: String, required: true },
7    isAdmin: { type: Boolean, default: false },
8  }, { timestamps: true });
9
10 module.exports = mongoose.model('User', userSchema);
11
```

Booking Model (booking.js)

The **booking.js** file defines the structure of booking data stored in the database. It is created using **Mongoose**, which interacts with **MongoDB**.

Purpose

The Booking model is used to:

- Store details of movie ticket bookings or orders
- Link users with movies/shows
- Track booking status and transaction details

Common Fields

- **user** – Reference to the user who made the booking
- **movie/show** – Reference to the booked movie or show
- **seats** – Number or list of seats booked
- **totalPrice** – Total cost of the booking
- **status** – Booking status (confirmed/cancelled)
- **bookingDate** – Date and time of booking

```
JS Booking.js ×
backend > models > JS Booking.js > ...
1  const mongoose = require('mongoose');
2
3  const bookingSchema = new mongoose.Schema({
4    user: { type: mongoose.Schema.Types.ObjectId, required: true },
5    show: { type: mongoose.Schema.Types.ObjectId, ref: 'Show', required: true },
6    seats: [{ type: String, required: true }],
7    totalPrice: { type: Number, required: true }
8  }, { timestamps: true });
9
10 module.exports = mongoose.model('Booking', bookingSchema);
11
```

Movie Model (movies.js)

The **movies.js** file defines the structure of movie data stored in the database. It is created using **Mongoose**, which interacts with **MongoDB**.

Purpose

The Movie model is used to:

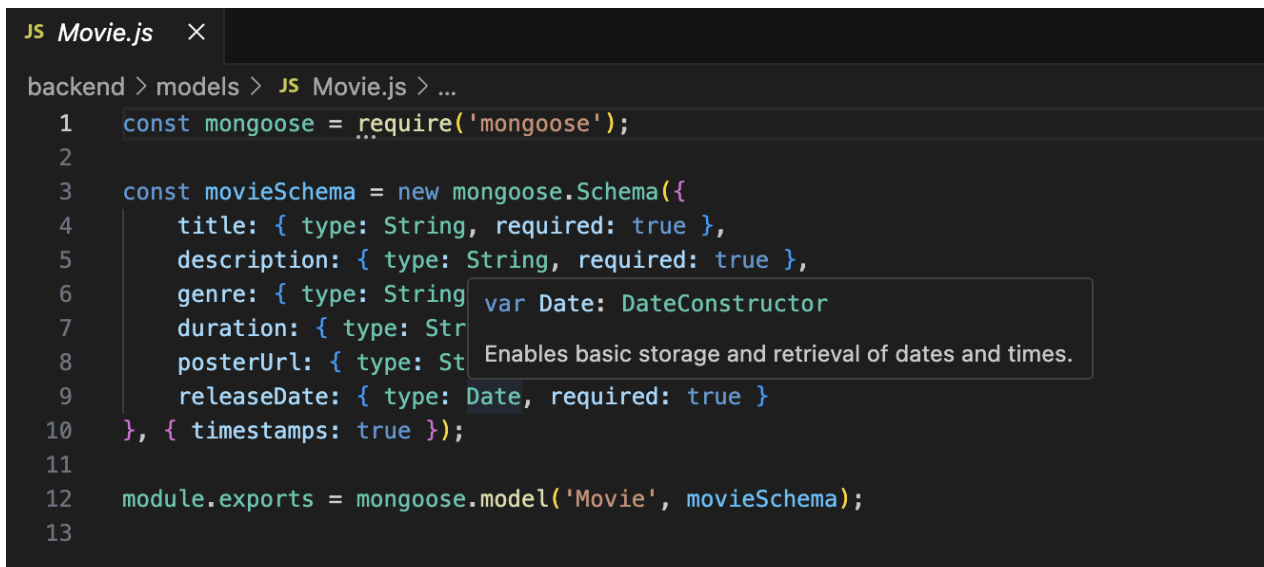
- Store information about movies available in the system
- Provide movie details for users to browse and book tickets
- Support show scheduling and booking processes

Schema Definition

The schema defines how movie data is structured in the database.

Common Fields

- **title** – Name of the movie
- **genre** – Category (Action, Drama, Comedy, etc.)
- **duration** – Length of the movie (in minutes)
- **language** – Language of the movie
- **releaseDate** – Movie release date
- **description** – Brief details about the movie
- **poster** – URL or path of the movie poster image



The screenshot shows a code editor with a file named `Movie.js`. The code defines a Mongoose schema for movies. It includes fields for title, description, genre, duration, posterUrl, and releaseDate. A tooltip for the `Date` type indicates it enables basic storage and retrieval of dates and times.

```
JS Movie.js ×
backend > models > JS Movie.js > ...
1  const mongoose = require('mongoose');
2
3  const movieSchema = new mongoose.Schema({
4    title: { type: String, required: true },
5    description: { type: String, required: true },
6    genre: { type: String, required: true },
7    duration: { type: String, required: true },
8    posterUrl: { type: String, required: true },
9    releaseDate: { type: Date, required: true }
10 }, { timestamps: true });
11
12 module.exports = mongoose.model('Movie', movieSchema);
13
```

var Date: DateConstructor
Enables basic storage and retrieval of dates and times.

Show Model (shows.js)

The **shows.js** file defines the structure of show-related data in the database. It is created using **Mongoose**, which works with **MongoDB**.

Purpose

The Show model is used to:

- Manage movie show timings
- Link movies with theaters/screens
- Track seat availability for each show
- Support the booking process

Schema Definition

The schema defines how show data is structured.

Common Fields

- **movie** – Reference to the movie
- **theaterName** – Name of the theater
- **screen** – Screen number or name
- **showTime** – Date and time of the show
- **totalSeats** – Total number of seats available
- **bookedSeats** – List of already booked seats

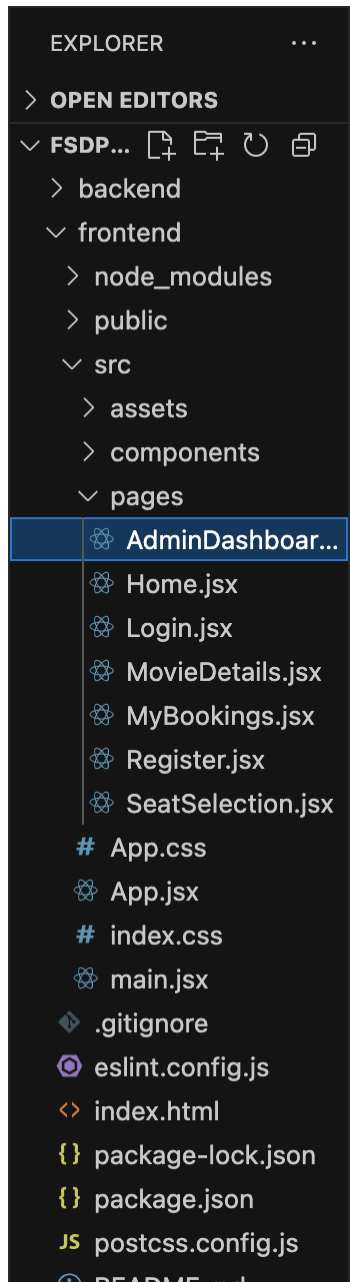
```
JS Show.js ×
backend > models > JS Show.js > ...
1  const mongoose = require('mongoose');
2
3  const showSchema = new mongoose.Schema({
4    movie: { type: mongoose.Schema.Types.ObjectId, ref: 'Movie', required: true },
5    date: { type: Date, required: true },
6    time: { type: String, required: true },
7    price: { type: Number, required: true },
8    bookedSeats: { type: [String], default: [] }
9  }, { timestamps: true });
10
11 module.exports = mongoose.model('Show', showSchema);
12
```

FRONT-END DEVELOPMENT:

Frontend Reference Video:

<https://drive.google.com/file/d/11Wk5HUPbrE50GnzE5KQPEaAbZUAEKi6S/view?usp=sharing>

Frontend Structure:



Admin Dashboard.jsx

Purpose

- Admin control panel for managing the system
- Allows admin to handle movies, shows, and bookings

Main Functionalities

1. Movie Management

- Stores movie data using useState
 - title
 - description
 - genre
 - duration
 - posterUrl
 - releaseDate
- Fetches movies from backend using axios
- Displays and manages movie list

2. Booking Management

- Uses bookings state
- Fetches all booking details
- Helps admin monitor user bookings

3. Show Management

- Uses states like:
 - showMovieId
 - showDate
 - showTime
 - showPrice
- Used to create and manage shows

2. Home.jsx

- Main landing page
- Displays list of movies
- Shows movie posters, titles, basic details
- Allows users to browse and select movies

3. Login.jsx

- Used for user authentication
- Takes email and password
- Sends request to backend
- Stores user data in localStorage after login

4. Register.jsx

- Used for new user registration
- Collects user details (name, email, password)
- Sends data to backend
- Creates new account

5. MovieDetails.jsx

- Displays full details of selected movie
- Shows:
 - Title
 - Description
 - Genre
 - Duration
 - Poster
- Allows user to proceed to booking

6. SeatSelection.jsx

- Used to select seats for a show
- Displays seat layout
- Shows available and booked seats
- Allows user to choose seats
- Calculates total price

7. MyBookings.jsx

- Displays user booking history
- Shows:
 - Movie name
 - Date & time
 - Seats booked
 - Payment status
- Allows users to view their tickets

Components

The **components** folder contains reusable UI elements used across the frontend of the application. These components help in building a modular, maintainable, and scalable user interface.

Purpose of Components

- Break the UI into **small reusable parts**
- Improve **code reusability and readability**
- Maintain a **clean and organized structure**
- Enable easy updates and maintenance

Navbar.jsx

This component represents the **navigation bar** of the application.

Key Features

- Provides navigation links to different pages (Home, Movies, Bookings, etc.)
- Displays user options like login/logout
- Ensures smooth navigation across the application
- May include logo, menu items, and user profile options

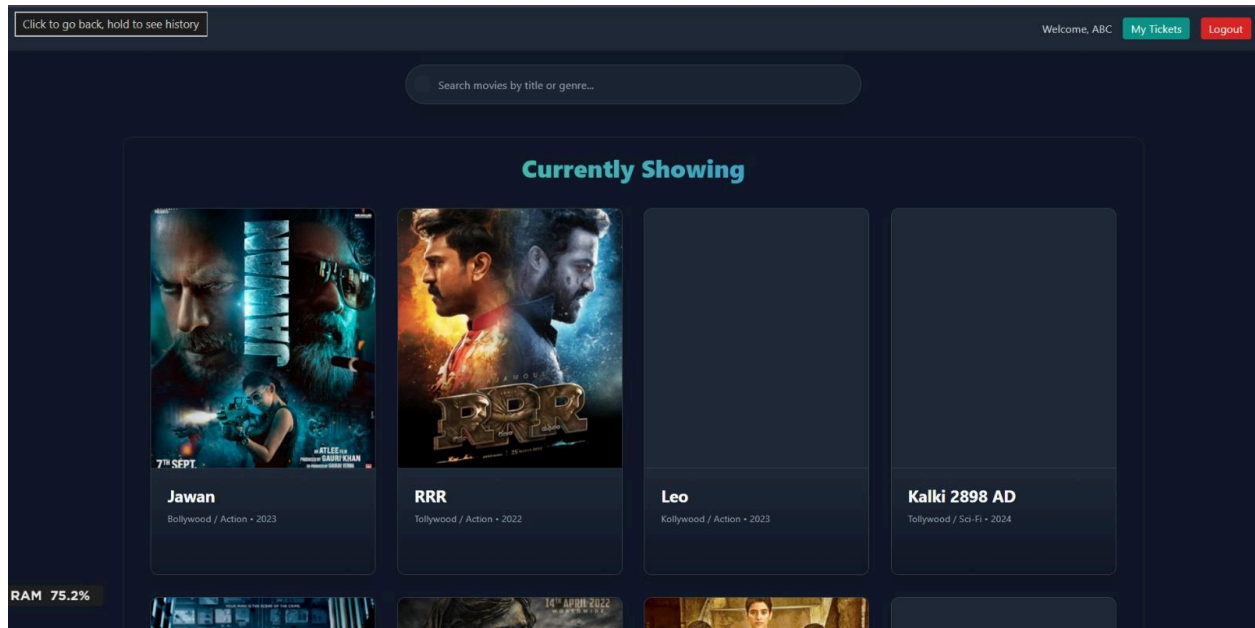
Role in Frontend Development

- Components are used inside pages to build the UI
- They interact with backend APIs to display dynamic data
- Help in maintaining a consistent design across the application

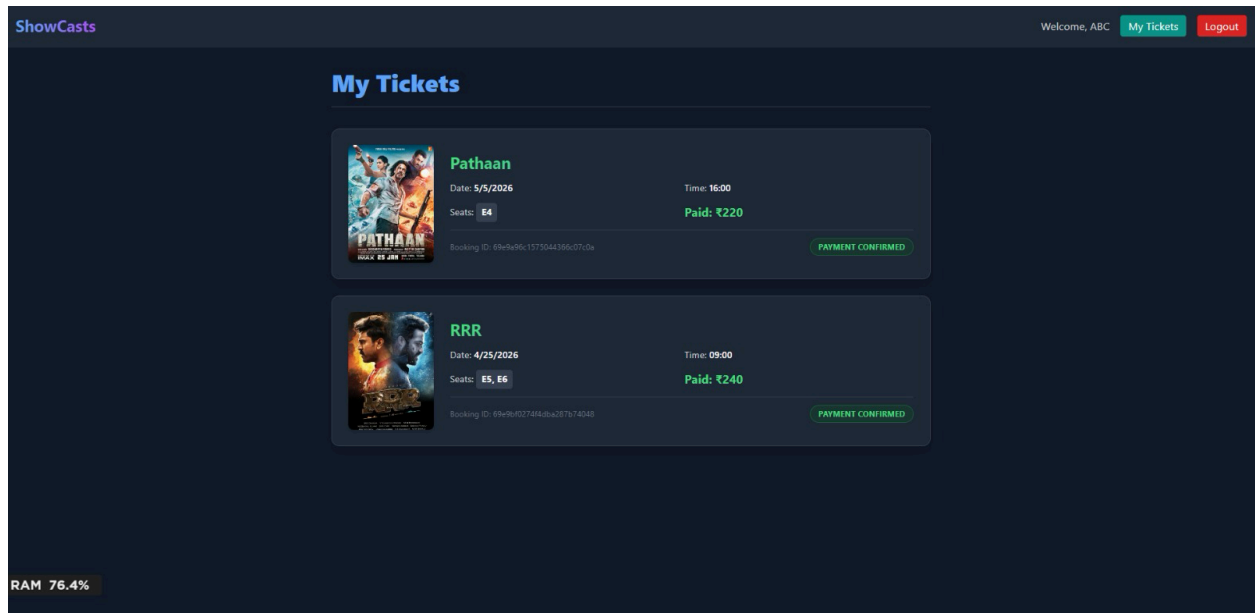
```
Navbar.jsx
frontend > src > components > Navbar.jsx > ...
1  import { Link, useNavigate } from 'react-router-dom';
2
3  const Navbar = () => {
4    const navigate = useNavigate();
5    const user = JSON.parse(localStorage.getItem('user'));
6
7    const handleLogout = () => {
8      localStorage.removeItem('user');
9      navigate('/login');
10   };
11
12   return (
13     <nav className="flex items-center justify-between p-4 bg-gray-800 text-white sticky top-0 z-50">
14       <Link to="/" className="text-2xl font-bold bg-clip-text text-transparent bg-gradient-to-r from-blue-400 to-indigo-600">
15         ShowCasts
16       </Link>
17       <div className="space-x-4 flex items-center">
18         {user ? (
19           <span className="text-gray-300">Welcome, {user.name}</span>
20           <Link to="/my-bookings" className="px-3 py-1 rounded bg-teal-600 hover:bg-teal-500 transition">My Bookings</Link>
21           {user.isAdmin && <Link to="/admin" className="px-3 py-1 rounded bg-indigo-600 hover:bg-indigo-500 transition">Admin</Link>}
22           <button onClick={handleLogout} className="px-3 py-1 rounded bg-red-600 hover:bg-red-500 transition">Logout</button>
23         ) : (
24           <Link to="/login" className="hover:text-blue-400 transition">Login</Link>
25           <Link to="/register" className="px-4 py-2 rounded bg-blue-600 hover:bg-blue-500 transition">Register</Link>
26         )}
27       </div>
28     </nav>
29   );
30
31   export default Navbar;
```

Output ScreenShots:

Home page:



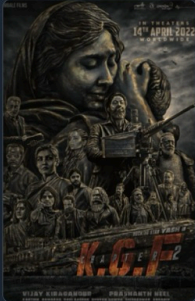
Booking tickets



Choose show timing

Click to go back, hold to see history

Welcome, ABCMy TicketsLogout



K.G.F: Chapter 2

In the blood-soaked Kolar Gold Fields, Rocky's name strikes fear into his foes. While his allies look up to him, the government sees him as a threat to law and order.

Genre: Sandalwood / Action

Duration: 168 mins

Release: 4/14/2022

Choose Show Timing

Thu, April 23

09:00

Tickets: ₹120

12:30

Tickets: ₹180

16:00

Tickets: ₹220

19:30

Tickets: ₹280

22:15

Tickets: ₹300

RAM 75.8%

Seats booking

ShowCasts

Welcome, ABCMy TicketsLogout

Screen This Way

A

1

2

3

4

5

6

7

8

9

10

B

1

2

3

4

5

6

7

8

9

10

C

1

2

3

4

5

6

7

8

9

10

D

1

2

3

4

5

6

7

8

9

10

E

1

2

3

4

5

6

7

8

9

10

F

1

2

3

4

5

6

7

8

9

10

G

1

2

3

4

5

6

7

8

9

10

H

1

2

3

4

5

6

7

8

9

10

RAM 76.5%

Available

Booked

Selected

Booking Summary

Seats (1):

F5

Total Price:

₹220

Proceed to Payment

Payment process

Secure Payment

Amount Due:

₹220

UPI

Card

Net Banking

Enter your UPI ID

₹ username@upi

A payment request will be sent to your UPI app. Please approve within 5 minutes.

Cancel

Pay & Book Now

Payment successful

localhost:5173 says

Payment via UPI accepted & Booking Successful!

OK

My Tickets



Pathaan

Date: 5/5/2026

Time: 16:00

Seats: **E4**

Paid: ₹220

Booking ID: 69e9a96c1575044366c07c0a

PAYMENT CONFIRMED



RRR

Date: 4/25/2026

Time: 09:00

Seats: **E5, E6**

Paid: ₹240

Booking ID: 69e9bf0274f4dba287b74048

PAYMENT CONFIRMED



RRR

Date: 4/23/2026

Time: 16:00

Seats: **F5**

Paid: ₹220

Booking ID: 69eb2e6422f7995f8324f0c8

PAYMENT CONFIRMED

Login

ShowCasts

LoginRegister

Login

Email

abc@example.com

Password

.....

Login

RAM 76.3%

Admin can add new movies

ShowCasts

Welcome, System Admin

My Tickets

Admin

Logout

Add New Movie

Title

Description

Genre

Duration (e.g. 120 mins)

Poster URL

dd-mm-yyyy

Add Movie

Add New Show

Select Movie

dd-mm-yyyy

--:--

Price (₹)

Add Show

RAM 77.0%

Admin can delete movie

ShowCasts

Welcome, System Admin

My Tickets

Admin

Logout

Manage Movies

Title	Genre	Release Date	Actions
Jawan	Bollywood / Action	9/7/2023	Delete
RRR	Tollywood / Action	3/24/2022	Delete
Leo	Kollywood / Action	10/19/2023	Delete
Kalki 2898 AD	Tollywood / Sci-Fi	5/9/2024	Delete
Inception	Hollywood / Sci-Fi	7/16/2010	Delete
K.G.F: Chapter 2	Sandalwood / Action	4/14/2022	Delete
Dangal	Bollywood / Drama	12/23/2016	Delete
Vikram	Kollywood / Thriller	6/3/2023	Delete

RAM 77.2%

Manage booking

ShowCasts

Welcome, System Admin

My Tickets

Admin

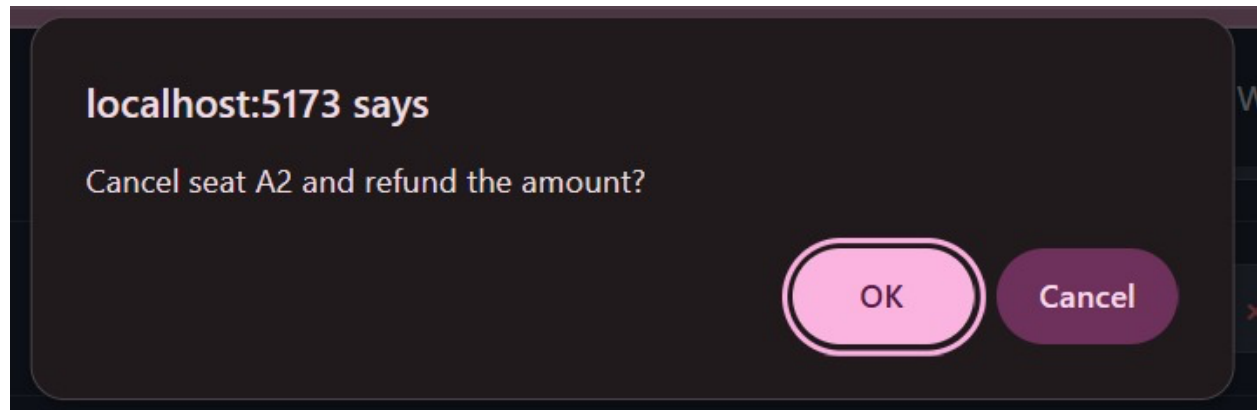
Logout

Manage Bookings

User	Movie	Show Date/Time	Seats	Paid
Bhargavi bharothubhargavi12@gmail.com	Jawan	4/23/2026 @ 09:00	A1 ✕ A2 ✕	₹240
Bhargavi bharothubhargavi12@gmail.com	Jawan	4/23/2026 @ 16:00	C4 ✕	₹220
Bhargavi bharothubhargavi12@gmail.com	RRR	4/23/2026 @ 16:00	C4 ✕	₹220
XYZ xyz@example.com	RRR	4/23/2026 @ 19:30	D5 ✕ D6 ✕	₹560
Bhargavi bharothubhargavi12@gmail.com	Kalki 2898 AD	4/23/2026 @ 19:30	D5 ✕ D6 ✕	₹560
XYZ xyz@example.com	Inception	4/23/2026 @ 19:30	D5 ✕ D6 ✕	₹560
XYZ xyz@example.com	Vikram	4/23/2026 @ 09:00	A1 ✕ A2 ✕	₹240

RAM 77.4%

Cancellation of seat



After cancellation the money refund



Demo Video Link:

https://drive.google.com/file/d/1jA32HXdgzFYowcpQT7lrXLPErxztnYtm/view?usp=drive_link

Code Drive Link:

https://drive.google.com/drive/folders/1h9_aCqGotFwk2dOKaYfclxUvA6mDjFvn